Machine Learning: Science and Technology

ACCEPTED MANUSCRIPT • OPEN ACCESS

Quantum computing model of an artificial neuron with continuously valued input data

To cite this article before publication: Stefano Mangini *et al* 2020 *Mach. Learn.: Sci. Technol.* in press https://doi.org/10.1088/2632-2153/abaf98

Manuscript version: Accepted Manuscript

Accepted Manuscript is "the version of the article accepted for publication including all changes made as a result of the peer review process, and which may also include the addition to the article by IOP Publishing of a header, an article ID, a cover sheet and/or an 'Accepted Manuscript' watermark, but excluding any other editing, typesetting or other changes made by IOP Publishing and/or its licensors"

This Accepted Manuscript is **© 2020 The Author(s). Published by IOP Publishing Ltd**.

As the Version of Record of this article is going to be / has been published on a gold open access basis under a CC BY 3.0 licence, this Accepted Manuscript is available for reuse under a CC BY 3.0 licence immediately.

Everyone is permitted to use all or part of the original content in this article, provided that they adhere to all the terms of the licence https://creativecommons.org/licences/by/3.0

Although reasonable endeavours have been taken to obtain all necessary permissions from third parties to include their copyrighted content within this article, their full citation and copyright line may not be present in this Accepted Manuscript version. Before using any content from this article, please refer to the Version of Record on IOPscience once published for full citation and copyright details, as permissions may be required. All third party content is fully copyright protected and is not published on a gold open access basis under a CC BY licence, unless that is specifically stated in the figure caption in the Version of Record.

View the article online for updates and enhancements.

This content was downloaded from IP address 212.70.14.214 on 15/08/2020 at 02:41

Quantum computing model of an artificial neuron with continuously valued input data

Stefano Mangini,[1,*] Francesco Tacchino,[1,†] Dario Gerace,[1] Chiara Macchiavello,[1,2,3] and Daniele Bajoni[4]

[1] *Dipartimento di Fisica, Università di Pavia, via Bassi 6, I-27100, Pavia, Italy*
[2] *INFN Sezione di Pavia, via Bassi 6, I-27100, Pavia, Italy*
[3] *CNR-INO, largo E. Fermi 6, I-50125, Firenze, Italy*
[4] *Dipartimento di Ingegneria Industriale e dell'Informazione, Università di Pavia, via Ferrata 1, I-27100, Pavia, Italy*

(Dated: July 31, 2020)

Artificial neural networks have been proposed as potential algorithms that could benefit from being implemented and run on quantum computers. In particular, they hold promise to greatly enhance Artificial Intelligence tasks, such as image elaboration or pattern recognition. The elementary building block of a neural network is an artificial neuron, i.e. a computational unit performing simple mathematical operations on a set of data in the form of an input vector. Here we show how the design for the implementation of a previously introduced quantum artificial neuron [npj Quant. Inf. **5**, 26], which fully exploits the use of superposition states to encode binary valued input data, can be further generalized to accept continuous- instead of discrete-valued input vectors, without increasing the number of qubits. This further step is crucial to allow for a direct application of gradient descent based learning procedures, which would not be compatible with binary-valued data encoding.

I. INTRODUCTION

Quantum computers hold the promise to greatly enhance the computational power of not-so-distant in future computing machines [1, 2]. In particular, improving machine learning techniques by means of quantum computers is the essence of the raising field of the field of Quantum Machine Learning [3–5]. Several models for the quantum computing version of artificial neurons have been proposed [6–12], together with novel quantum machine learning techniques implementing classification tasks [13–15], quantum autoencoders [16, 17], quantum convolutional networks [18, 19] and Quantum Boltzmann Machines [20, 21] to give a non-exhaustive list.

In this context, quantum signal processing leverages the capabilities of quantum computers to represent and elaborate exponentially large arrays of numbers, and it could be used for enhanced pattern recognition tasks, i.e. going beyond the capabilities of classical computing machines [22]. In these regards the development of Neural Networks dedicated for quantum computers [23] is of fundamental importance, due to the preponderance of this type of classical algorithms in image processing [24, 25].

In the commonly accepted terminology of graph theory, neural networks are directed acyclic graphs (DAG), i.e., a collection of nodes where information flows only in one direction, without any loop. Each node is generally defined an artificial neuron, i.e., a simplified mathematical model of natural neurons. In practice, it consists of an object function that takes some input data, processes them using some internal parameters (defined weights), and eventually gives an output value. In their simplest form, the so called McCulloch-Pitts neurons [26] only deal with binary values, while in the most common and most useful form, named perceptron [27], they accept real, continuously valued inputs and weights.

Continuous inputs are not possible in conventional, digital computers, and these are usually rendered by using bit strings: a grey scale image pixel is for instance usually rendered in natural numbers on a scale from 0 to 255 using 8-bit binary strings. Some approaches propose to use a similar representation in quantum computers by assigning several qubits per value [28–30]. However, these approaches are particularly wasteful, especially in light of the fact that quantum mechanical wavefunctions can be inherently represented as continuously valued vectors.

A previous work [9] introduced a model for a quantum circuit mimicking a McCulloch-Pitts neuron. Here we generalize that model to the case of a quantum circuit accepting also continuously valued input vectors. We thus present a model for a continuous quantum neuron which, as we will see, can be used for pattern recognition in greyscale images without the need to increase the number of qubits to be employed. This represents a further memory advantage with respect to classical computation, where an increase in the number of encoding bits is required to deal with continuous numbers. We employ a phase-based encoding, and show that it is particularly resilient to noise.

Differently from classical perceptron models, artificial quantum neurons as described, e.g., in Ref. [9] can be used to classify linearly non separable sets. In the continuously valued case, we thus harness the behaviour of our quantum perceptron model to show its ability to correctly classify several notable cases of linearly non separable sets. Furthermore, we test this quantum artificial neuron for digit recognition on the MNIST dataset [31], with remarkably good results. We further stress that the present generalization of the binary-valued artificial

* stefano.mangini01@universitadipavia.it
† Present address *IBM Research GmbH, Zurich Research Laboratory, Säumerstrasse 4, CH-8803 Rüschlikon, Switzerland*

Accepted Manuscript

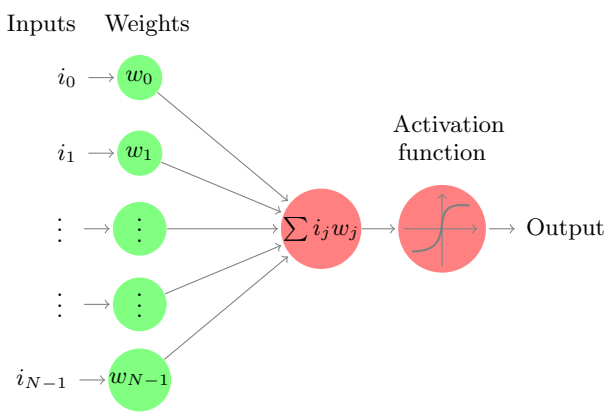


FIG. 1: **Scheme of a classical perceptron model.** The artificial neuron evaluates a weighted sum between the input vector, $\vec{i}$, and the weight vector, $\vec{w}$, followed by an activation function, which determines the actual output of the neuron.

neuron model is a crucial step in view of fully exploiting the great potential allowed from automatic differentiation such as gradient descent. These techniques are commonly employed, e.g., in supervised and unsupervised learning procedures, and would be impossible to be applied to the oversimplified McCulloch-Pitts neuron model.

II. CONTINUOUSLY VALUED QUANTUM NEURON MODEL

A. The algorithm

Let us consider a perceptron model with real valued input and weight vectors, which are respectively indicated as $\vec{i}$ and $\vec{w}$, such that $i_j, w_j \in \mathbb{R}$. A schematic representation of the classical perceptron is reported in Fig. 1.

Similarly, we define a model of a quantum neuron capable of accepting continuously valued input and weight vectors, by extending a previous proposal for the quantum computing model of an artificial neuron only accepting binary valued input data [9]. In order to encode data on a quantum state, we make use of a phase encoding. Given an input $\boldsymbol{\theta} = (\theta_0, \ldots, \theta_{N-1})$ with $\theta_i \in [0, \pi]$, which consists of the classical data to be analyzed, we consider the vector:

$$\vec{i} = (e^{i\theta_0}, e^{i\theta_2}, \cdots, e^{i\theta_{N-1}}) \ , \tag{1}$$

which we will be referring to as the input vector in the following. For data not lying in the interval $[0, \pi]$ but more generally in $[a, b]$, a normalization scheme can be used, such that $\theta_i \to \frac{\theta_i - a}{b-a}\pi$, allowing to transform the data in the appropriate range. Explicit examples will be given later. With the input vector in Eq. (1) we define the corresponding input quantum state of $n = \log_2 N$ qubits:

$$|\psi_i\rangle = \frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} i_k \, |k\rangle \ , \tag{2}$$

where the states $|k\rangle$ denote the computational basis states of n qubits ordered by increasing binary representation, $\{|00\ldots0\rangle, |00\ldots1\rangle, \cdots, |11\ldots1\rangle\}$. Since we are dealing with an artificial neuron, we have to properly encode another vector, which represents the weights in the form $\boldsymbol{\phi} = (\phi_0, \ldots, \phi_{N-1})$ with $\phi_i \in [0, \pi]$, i.e. the corresponding vector:

$$\vec{w} = (e^{i\phi_0}, e^{i\phi_2}, \cdots, e^{i\phi_{N-1}}) \tag{3}$$

which in turn defines the weight quantum state:

$$|\psi_w\rangle = \frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} w_k \, |k\rangle \ . \tag{4}$$

Notice that (2) and (4) have the same structure, i.e. they consist of an equally weighted superposition of all the computational basis states, although with varying phases. By means of such encoding scheme, we can fully exploit the exponentially large dimension of the n qubits Hilbert space, i.e., by only using n qubits it is evidently possible to encode and analyze data of dimension $N = 2^n$. Due to global phase invariance, the number of actual independent phases is $2^n - 1$, which does not spoil the overall efficiency of the algorithm, as it will be shown. We also notice that the class of states represented as $\frac{1}{2^{n/2}} \sum_i e^{i\alpha_i} |i\rangle$, as (2) and (4) are known as locally maximally entanglable (LME) states, as introduced in Ref. [32].

Having defined the input and weight quantum states, their similarity is estimated by considering the inner product

$$\begin{aligned} \langle\psi_w|\psi_i\rangle &:= \frac{1}{2^n} \sum_{k,j=0}^{2^n-1} i_k w_j^* \, \langle j|k\rangle \\ &= \frac{1}{2^n} \, \vec{i} \cdot \vec{w}^* \\ &= \frac{1}{2^n} \left(e^{i(\theta_0 - \phi_0)} + \cdots + e^{i(\theta_{2^n-1} - \phi_{2^n-1})} \right) \ , \end{aligned} \tag{5}$$

which corresponds to evaluating the scalar product between the input vector in Eq. (1) and the conjugated of the weight vector in Eq. (3), $\vec{w}^*$, similarly to the classical perceptron algorithm. Since probabilities in quantum mechanics are represented by the squared modulus of wavefunction amplitudes, we consider $|\langle\psi_w|\psi_i\rangle|^2$, which is explicitly given as (see App. A):

$$|\langle\psi_w|\psi_i\rangle|^2 = \frac{1}{2^n} + \frac{1}{2^{2n-1}} \sum_{i<j}^{2^n-1} \cos((\theta_j - \phi_j) - (\theta_i - \phi_i)) \ . \tag{6}$$

It is easily checked that $|\langle\psi_w|\psi_i\rangle|^2 = 1$ for $\theta_i = \phi_i \ \forall i$, since the two states would coincide in such case.

Accepted Manuscript

Equation (6) represents the *activation function* implemented by the proposed quantum neuron. Even if it does not remind any of the activation functions conventionally used in classical machine learning techniques, such as the Sigmoid or ReLu functions [33], its nonlinearity suffices to accomplish classification tasks, as we will discuss in the following sections.

Color invariance and noise resilience

From Eq. (6), we define the activation function of the quantum artificial neuron as

$$f(\boldsymbol{\theta}, \boldsymbol{\phi}) = |\langle \psi_w | \psi_i \rangle|^2 \ . \tag{7}$$

Keeping $\boldsymbol{\phi}$ fixed, suppose two different input vectors are passed to the quantum neuron: $\boldsymbol{\theta}$ and $\boldsymbol{\theta'} = \boldsymbol{\theta} + \boldsymbol{\Delta}$, with $\boldsymbol{\Delta} = (\Delta, \ldots, \Delta)$. Whatever the value of Δ, it is easy to infer that both input vectors will result in the same activation function. Hence, two input vectors only differing by a constant, albeit real valued, quantity will be equally classified by such model of quantum perceptron. Hence, in the context of image classification, we can state that the present algorithm has a built in color translational invariance. This should not come as a surprise, since the activation function actually depends of the *differences* between phases. In fact, the artificial neuron tends to recognize as similar any dataset that displays the same overall differences, instead of perfectly coincident datasets.

Next, we assume that the input and weight vectors do coincide, but only up to some noise corrupting the input vector, such that: $\boldsymbol{\theta} = \boldsymbol{\phi} + \boldsymbol{\Delta}$, where $\boldsymbol{\Delta} = (\Delta_0, \Delta_1, \ldots, \Delta_{2^n-1})$ represents the small variations, now assumed to be different on each pixel. Substituting the above values in Eq. (7), we obtain

$$f(\boldsymbol{\theta}, \boldsymbol{\phi}) = f(\boldsymbol{\Delta}) = \frac{1}{2^n} + \frac{1}{2^{2n-1}} \sum_{i<j}^{2^n-1} \cos(\Delta_j - \Delta_i) \ . \tag{8}$$

For simplicity of calculation, we assume the noise factors, Δ_i, to be distributed according to a uniform distribution in the interval $[-a/2, a/2]$, the activation function averaged over the probability distribution of Δ_i can be calculated as (see App. B):

$$\langle f(\boldsymbol{\Delta}) \rangle = \frac{1}{2^n} + \frac{2^n - 1}{2^{n-1}} \left(\frac{1 - \cos(a)}{a^2} \right) \ . \tag{9}$$

Since all the possible input data lie in the interval $[0, \pi]$, a reasonable noise would be of the order of some small fraction of π, which implies $a < 1$. Hence, in the case of small noise, Eq. (9) can be recast as

$$\langle f(\boldsymbol{\Delta}) \rangle = 1 - \frac{2^n - 1}{2^n} \frac{a^2}{12} + O\left(a^6\right) \qquad \text{for } a \ll 1 \ . \tag{10}$$

Thus, the classification of the quantum neuron is only slightly perturbed by the presence of noise corrupting an input vector otherwise having a perfect activation. By similar calculations, it can be shown that this property also holds for any kind of input vector, not only those with perfect activation (see App. B). With a more a realistic noise model of the errors given as a Gaussian distribution, similar results have been obtained.

After having outlined the main steps defining the quantum perceptron model for continuously valued input vectors, we now proceed to build a quantum circuit that allows implementing it on a qubit-based quantum computing hardware.

B. Quantum circuit model of a continuously valued perceptron

A quantum circuit implementing the quantum neuron described above is schematically represented in Fig. 2. The first section of the circuit, denoted as U_i, transforms the quantum register, initialized into the reference state $|0\rangle^{\otimes n}$, to the input quantum state defined in Eq. (2); the following operation U_w performs the inner product between the input and weight quantum state; finally, a multi-controlled CNOT targeting an ancillary qubit is used to extract the final result of the computation. We now explain in detail how these transformations can be achieved.

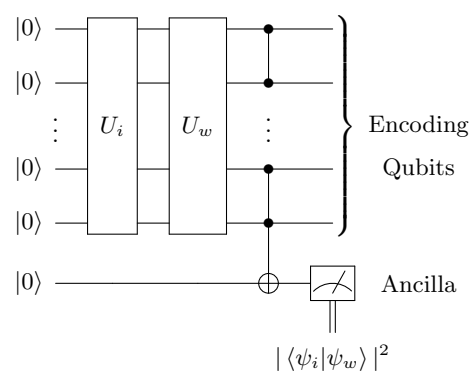


FIG. 2: Quantum circuit model of a perceptron with continuously valued input and weight vectors.

Starting from the n-qubit state, $|00\ldots0\rangle = |0\rangle^{\otimes n}$, the U_i operation creates the quantum input state $U_i |0\rangle^{\otimes n} = |\psi_i\rangle$ (2). Such a unitary can be built by means of a brute force approach. First of all, we apply a layer of Hadamard gates, $\mathrm{H}^{\otimes n}$, which creates the balanced superposition state $\mathrm{H}^{\otimes n} |0\rangle^{\otimes n} = |+\rangle^{\otimes n}$, with $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$. The quantum state $|+\rangle^{\otimes n}$ consists of an equally weighted superposition of all the states in the n qubits computational basis, hence we can target each of them and add the appropriate phase to it, in order to obtain the desired result. This action corresponds to the diagonal (in the

Accepted Manuscript

computational basis) unitary operation

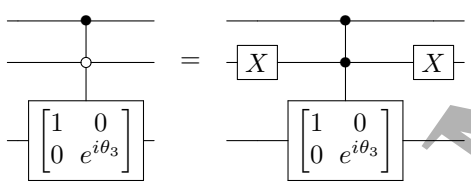
$$U(\boldsymbol{\theta}) := \begin{bmatrix} e^{i\theta_0} & 0 & \cdots & 0 \\ 0 & e^{i\theta_1} & \cdots & \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & e^{i\theta_{2^n-1}} \end{bmatrix} \tag{11}$$

whose action is to phase shift each state of the computational basis, $|i\rangle$, to $e^{i\theta_i}|i\rangle$, with phases $\theta_i \in \mathbb{R}$, that are (in general) independent from each other. We decompose

$$U(\boldsymbol{\theta}) = \prod_{i=0}^{2^n-1} U(\theta_i) \ ,$$

where $U(\theta_i)$ is the unitary whose action is $U(\theta_i)|i\rangle = e^{i\theta_i}|i\rangle$, while leaving all the other states in the computational basis unchanged. These unitaries are equivalent to a combination of X gates and a multi-controlled phase shift gate, $\mathrm{C}^{n-1}\mathrm{R}(\theta)$, where the phase shift gate is the unitary operation defined as $\mathrm{R}(\theta) = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{bmatrix}$ [34].

For example, suppose having $n = 3$ qubits, and consider the state $|101\rangle$ to be phase shifted to $e^{i\theta_3}|101\rangle$. This transformation is achieved by the following quantum circuit:



which implements the desired transformation $U(\theta_3)|101\rangle = e^{i\theta_3}|101\rangle$, while leaving all other states of the computational basis unchanged. Iterating a similar gate sequence for each state of the computational basis eventually yields the overall unitary operation, (11). So far, we have built the quantum circuit allowing to encode an arbitrary input vector: given the input $\vec{i} = (e^{i\theta_0}, e^{i\theta_2}, \cdots, e^{i\theta_{2^n-1}})$ as in Eq. (1), we create the state $|\psi_i\rangle$ (2), by means of the operation $U_i|0\rangle^{\otimes n} = U(\boldsymbol{\theta})\mathrm{H}^{\otimes n}|0\rangle^{\otimes n} = |\psi_i\rangle$, whose parameters depend on the input entries.

The unitary U_w can then be constructed in a similar fashion. First of all, notice that the U_i is unitary, thus reversible. Be $\vec{w} = (e^{i\phi_0}, e^{i\phi_2}, \cdots, e^{i\phi_{2^n-1}})$ the weight vector, then the desired inner product $\langle\psi_w|\psi_i\rangle$ (6) resides in the overlap of the quantum state $|\phi_{i,w}\rangle = (U(\boldsymbol{\phi})\mathrm{H}^{\otimes n})^\dagger|\psi_i\rangle$ with the ground state $|0\ldots0\rangle$. In fact, since $U(\boldsymbol{\phi})\mathrm{H}^{\otimes n}|0\rangle^{\otimes n} = |\psi_w\rangle$ (4), the scalar product is clearly given as

$$\langle 0\ldots0|\overbrace{(U(\boldsymbol{\phi})\mathrm{H}^{\otimes n})^\dagger|\psi_i\rangle}^{|\phi_{i,w}\rangle} = \underbrace{\langle 0\ldots0|\mathrm{H}^{\otimes n}U(\boldsymbol{\phi})^\dagger}_{\langle\psi_w|}|\psi_i\rangle = \langle\psi_w|\psi_i\rangle \ . \tag{12}$$

In order to extract the result, a final layer of $\mathrm{X}^{\otimes n}$ gates is applied to all encoding qubits, such that the desired coefficient now multiplies the component $|1\rangle^{\otimes n}$ in the superposition:

$$\begin{aligned} \mathrm{X}^{\otimes n}|\phi_{i,w}\rangle &= |\bar{\phi}_{i,w}\rangle \\ &= \sum_{j=0}^{2^n-2} c_j|j\rangle + c_{2^n-1}|11\ldots1\rangle \end{aligned} \tag{13}$$

with $c_{2^n-1} = \langle\psi_w|\psi_i\rangle$. Thus, the U_w transformation in Fig. 2 actually consists in the quantum operations $U_w = \mathrm{X}^{\otimes n}\mathrm{H}^{\otimes n}U(\boldsymbol{\phi})^\dagger$.

By means of a multi-controlled $\mathrm{C}^n\mathrm{NOT}$, we load the result on an ancillary qubit

$$\mathrm{C}^n\mathrm{NOT}(|\bar{\phi}_{i,w}\rangle|0\rangle_a) = \sum_{j=0}^{2^n-1} c_j|j\rangle|0\rangle_a + c_{2^n-1}|11\cdots1\rangle|1\rangle_a \ . \tag{14}$$

Eventually, a final measurement of the ancilla qubit will yield result 1, which is interpreted as a firing neuron, with probability $|c_{2^n-1}|^2 = |\langle\psi_w|\psi_i\rangle|^2 = |\vec{i}\cdot\vec{w}^*|^2/(2^{2n})$, which consists in the neuron activation function, Eq. (6).

We notice that an input vector containing $N = 2^n$ elements only requires $n+1$ qubits to implement the quantum circuit above, one of them being the ancilla qubit. To avoid introducing an ancilla qubit, an alternative strategy would be to perform a joint measurement on all n qubits in the state $|\phi_{i,w}\rangle$ given in Eq. (12), with the probability of obtaining $|0\ldots0\rangle$ being proportional to the inner product. However, since machine learning techniques yield their full potential when used with connected structures of multiple single neurons, and having in mind the idea of implementing the quantum computing version of a feedforward neural network, it is essential to have a model for which information is easily transferred from each neuron to the following layer. This can be accomplished by using an ancilla qubit per artificial neuron, where the quantity of interest can be loaded [35]. The time complexity of this quantum circuit depends linearly on the dimension of the input vectors N. Indeed, the quantum circuit introduced above requires $O(N)$ operations to implement all the phase shifts necessary to build the LME states, like Eq. (2). Depending of the relation between the input data, θ_i, other preparation schemes involving less operations could be devised [32] (see Appendix C for a discussion of alternative schemes to implement the quantum neuron circuit). Finally, it is worth noticing that thanks to global phase invariance, the activation function (6) can be recast as:

$$\begin{aligned} |\langle\psi_w|\psi_i\rangle|^2 &= \frac{1}{2^{2n}}\left|\sum_{i=0}^{2^n-1} e^{i(\theta_i-\phi_i)}\right|^2 \\ &= \frac{1}{2^{2n}}\left|1 + \sum_{i=1}^{2^n-1} e^{i(\tilde{\theta}_i-\tilde{\phi}_i)}\right|^2 \ , \end{aligned} \tag{15}$$

Accepted Manuscript

with $\tilde{\theta}_i = \theta_i - \theta_0$, $\tilde{\phi}_i = \phi_i - \phi_0$. By exploiting this redefinition of the parameters, it is possible to implement the same transformation but employing fewer gates, since it is equivalent to leaving the state $|0\rangle^{\otimes n}$ unchanged during the whole computation. Depending on the actual quantum hardware and data, further simplifications to the circuit could be obtained in compiling time. In Fig. 3, the scheme of a quantum circuit implementing the artificial neuron model is shown for the specific case involving $n = 2$ qubits.

III. RESULTS: IMAGE RECOGNITION AND LEARNING

The quantum neuron model introduced above is an ideal candidate to perform classification tasks involving grayscale images. A grayscale image consists of a grid of pixels whose intensities are usually[1] represented by integer numbers in the range $[0, 255]$, as shown in Fig. 4.

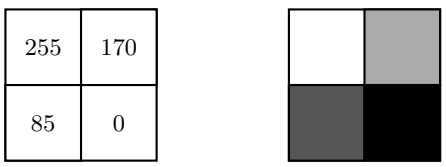


FIG. 4: A grayscale image with corresponding pixel intensities. This image can be encoded in the array $(255, 170, 85, 0)$, ordered from top-left to bottom-right.

Since we make use of a phase encoding, all inputs (and weights) to the artificial neuron have to be normalized in the interval $[0, 2\pi]$. In this work we further restrict this domain for two reasons. First, values in $[0, \pi]$ and $[\pi, 2\pi]$ are fully equivalent, due to the periodicity in phase and the squared modulus in Eq. (6); second, for the same reason, states with zero or π phase yield the same activation function, which in turn means that images with inverted colors (i.e., by exchanging white with black) would be recognized as equivalent by this perceptron model. Hence, to distinguish a given image from its negative, we further restrict the input and weight elements to lie in the range $[0, \pi/2]$. Thus, an image such as the one reported in Fig. 4 is subject to the normalization $(255, 170, 85, 0) \rightarrow \frac{\pi/2}{255}(255, 170, 85, 0)$, before using it as an input vector to be encoded into the quantum neuron model.

We implemented and tested the quantum circuit both on simulators and on real quantum hardware, by using the IBM Quantum Experience [2] and Qiskit [36]. The results are reported in the following.

[1] This encoding of grayscale images employs a single byte (i.e., 8 bits) per pixel on a classical computing register.

[2] https://quantum-computing.ibm.com/

A. Numerical Results

To better appreciate the potentialities of the continuously valued quantum neuron, we analyse its performance in recognizing similar images. We fix the weight vector to $\boldsymbol{\phi} = (\pi/2, 0, 0, \pi/2)$, which corresponds to the checkerboard pattern represented in the image of Fig. 5, and then generate a few random images to be used as inputs to the quantum neuron. For each input, the circuit is executed multiple times, thus building a statistics of the outcomes. With $m = 30$ random generated images, the results of the classification are depicted in Fig. 5, which includes the analytic results, the results of numerical simulations run on Qiskit QASM Simulator backend, and finally the results obtained by executing the quantum circuit on the ibmqx2-yorktown (accessed in March 2020) real device. Due to errors in the actual quantum processing device, the statistics of the outcome differ from either the simulated one or the analytic result. Nevertheless, the same overall behaviour can be easily recognised, thus showing that the quantum neuron circuit can be successfully implemented also in an actual quantum processor giving reliable results for such recognition tasks. The images producing the largest activation are the ones corresponding to input vectors similar to the checkerboard-like weight vector, which confirms the desired behaviour of the quantum neuron in recognizing similar images. On the contrary, the images with lowest activation are similar to the negative of the target weight vector, as desired.

IV. LEARNING

The process of finding the appropriate value for the weights to implement a given classification is called *learning*, and it is generally based upon an optimization procedure in which a cost function is minimized by some gradient descent technique. Ideally, the minimum of the cost function corresponds to the targeted solution.

A simple learning task for our quantum neuron is to recognize a single given input. Starting from an input vector, $\boldsymbol{\theta}$, we aim at finding a weight vector, $\boldsymbol{\phi}$, producing a high activation. Since the activation function for our quantum neuron is given in Eq. (6), we know that perfect activation can only be obtained when the input and weight vectors are exactly coincident, $\boldsymbol{\theta} = \boldsymbol{\phi}$. This case can easily be checked numerically, by letting the neuron learn the right weights through a classical optimization technique.

A naive yet efficient choice for the cost function driving the learning process is $\mathcal{L}(\boldsymbol{\phi}) = (1 - f(\boldsymbol{\theta}, \boldsymbol{\phi}))^2$, in which $f(\boldsymbol{\theta}, \boldsymbol{\phi})$ is the activation function of the artificial neuron with input $\vec{\theta}$ and weight vector $\boldsymbol{\phi}$, as in Eq. (7). The minimum of the cost function, zero, is reached when the quantum neuron has full activation, i.e. $f(\boldsymbol{\theta}, \boldsymbol{\phi}) = 1$. The minimization process is driven by the Stochastic Perturbation Stochastic Approximations (SPSA) [37],

Accepted Manuscript

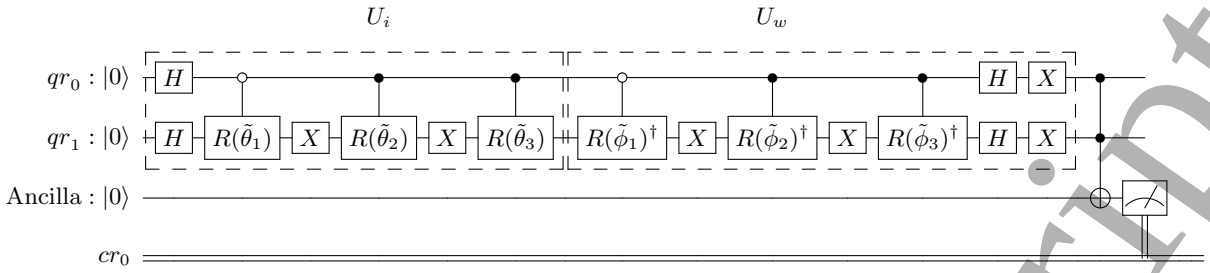


FIG. 3: Scheme of the quantum circuit for the $n = 2$ qubits case. The parameters are redefined as $\tilde{\theta}_i = \theta_i - \theta_0,\ \tilde{\phi}_i = \phi_i - \phi_0$, as detailed in Eq. (15).

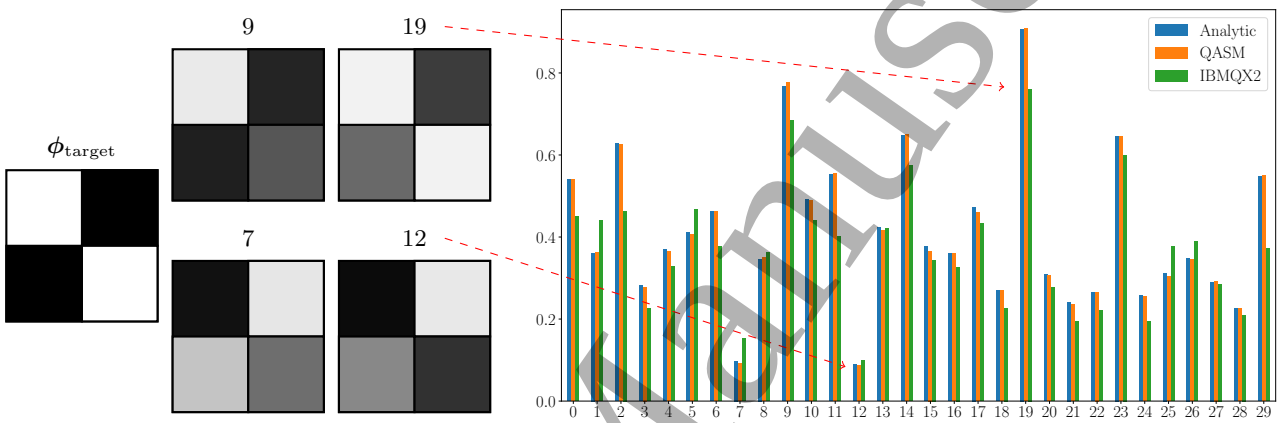


FIG. 5: Results of the image recognition task performed by a quantum artificial neuron, obtained by running the corresponding quantum circuit with either the Qiskit QASM Simulator backend or the ibmqx2-Yorktown real quantum processor. In addition, we also report the related analytic values. The target weight vector, $\boldsymbol{\phi}_{\text{target}}$, is fixed, and $m = 30$ random images are given as input vectors to the quantum artificial neuron. For each input, the corresponding quantum circuit is executed 8192 times. The checkerboard-like image corresponds to the target weight vector $\boldsymbol{\phi}_{\text{target}} = (\pi/2, 0, 0, \pi, 2)$, while the images displaying respectively largest and lowest activation are the ones labeled as 19 and 12. Input vectors labeled as 9 and 7 are examples of images with high and low activation, respectively.

which is built for optimization processes characterized by the presence of noise and is thus particularly effective in the presence of probabilistic measurement outputs.
An actual implementation on the QASM simulator leads to the following results. The task is to recognize the input vector $\vec{\theta} = (\pi/5,\ 0,\ \pi/3,\ 0.1)$. Using the SPSA optimizer, the cost function gets minimized by varying the weight vector, as reported in Fig. 6a, where it is evident that the cost function rapidly converges to values close to zero after a few iteration steps. The solution to the problem, that is the final optimized weight vector, is $\vec{\phi}_f = (1.03,\ 0.19,\ 1.47,\ 0.61)$, whose grayscale representation is plotted in Figure 6b. Even if the input and weight vector are not numerically equivalent, we can see that the final weight image actually looks very much like the target one, as expected. In fact, the two images retain almost the same shades of gray, with the optimized one being a bit shifted towards the brightest end of the spectrum, and as we previously noticed, the neuron is blind to overall color shifts.

In general, when dealing with a classification task, there is more than one input vector to be classified. Let us restrict ourselves to the case of a supervised binary classification[3], where a each input $\boldsymbol{\theta}$ is associated to a binary label, y, such that $y \in \{0, 1\}$. Thus, the learning procedure consists in finding the right parameters (i.e. a weight vector $\vec{w}$) for which the artificial neuron reproduces the correct association of a given input vector with its corresponding label. In order to implement this dichotomy in the perceptron model, it is common practice to introduce a *threshold* value, t: given an input and a weight vector, if the activation of the artificial neuron is

[3] This can be generalized in the case of a multi-class classification, by adopting a *one versus all* approach.

Accepted Manuscript

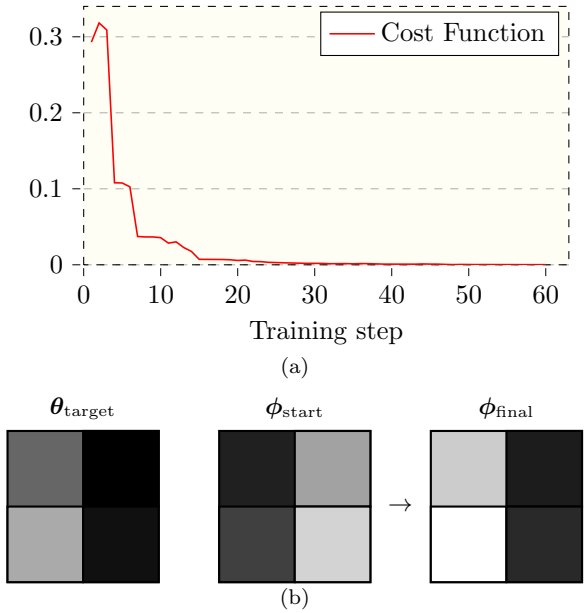


FIG. 6: (a) Minimization of the cost function, $\mathcal{L}(\boldsymbol{\phi}) = (1 - f(\boldsymbol{\theta}, \boldsymbol{\phi}))^2$, as a function of iteration steps. (b) Image corresponding to the target weight vector, $\boldsymbol{\theta}$ (left panel), and the weight vector before the optimization, $\boldsymbol{\phi}_0$ (center panel), and after the optimization, $\boldsymbol{\phi}_f$ (right panel).

above the value set by t, then the assigned label is 1; otherwise it is 0. Note that the the threshold t is actually a hyperparameter for our model, and in the following simulations it was heuristically optimized in order to achieve the best classification accuracy.

A common choice for the cost function is the distance of the correct label assignment from the one implemented by the artificial neuron, which is expressed as

$$\mathcal{L}(\boldsymbol{\phi}) = \frac{1}{M} \sum_{i=1}^{M} (y_i - \tilde{y}_i)^2 , \tag{16}$$

where M is the number of input entries, y_i is the correct label associated to input value $\boldsymbol{\theta}_i$, and $\tilde{y}_i$ is the label assigned by the neuron, which is calculated as

$$\tilde{y}_i = \begin{cases} 1 & \text{if } f(\boldsymbol{\theta}, \boldsymbol{\phi}) > t \\ 0 & \text{otherwise} \end{cases} . \tag{17}$$

The learning process then consists in minimizing the cost function, such as the one in Eq. (16).

Generally speaking, in a supervised learning procedure the inputs are divided into two distinct sets: the *training* set, which contains the input values that are used to drive the learning procedure, and the *test* set, which contains input vectors used to test the actual classification power of the quantum neuron with data never analysed before. Now that we have introduced the general learning framework, we can apply it to a few specific cases.

A. Learning of two dimensional data

As a first example, we consider a classification problem of the form $\{\boldsymbol{x}_i, y_i\}_{i=1,\dots,M}$, in which $\boldsymbol{x}_i = (x_1^i, x_2^i)$ are two dimensional input data, and y_i their labels, such as the ones represented in Fig. 7a. The color indicates the label associated to the input value, i.e., red for zero and blue for one. Since the data are two dimensional, we only need a single qubit to encode the information in the quantum state. The cost function (16) is minimized using the SPSA optimizer and its behaviour is reported in Fig. 7c. The learning procedure converges towards a minimum of the cost function, and its value on the test set displayed in Fig. 7b amounts to $\mathcal{L}_{\text{test}} = 0$. This can be seen in Fig. 7b, where we plot the decision boundary of the neuron along with the input values of the test set. All the calculations were performed on the QASM simulator.

B. Non separable points using a bias

We have just shown that a single neuron is sufficient to classify some kind of two dimensional data. The procedure might fail on more complex structures of the dataset, though. For example, if one needs to classify data as in Fig. 8a, a single qubit encoding of the quantum perceptron model is not enough. However, using a quantum neuron implemented with two qubits allows to capture more degrees of freedom, thus helping to successfully tackle the problem. In fact, with $n = 2$ qubits it is possible to encode $2^2 = 4$ parameters, or input data. Two of these are employed to encode the actual data of interest, one can be kept fixed to zero[4], and the last free parameter can be interpreted as a *bias*. Thus, a convenient encoding scheme is to consider input vectors of the form $\boldsymbol{\theta} = (\theta_0, \theta_1, \theta_2, \theta_3) = (0, x_1, x_2, 0)$, and weight vectors $\boldsymbol{\phi} = (\phi_0, \phi_1, \phi_2, \phi_3) = (0, \phi_1, \phi_2, b)$, where b denotes the bias. After the learning procedure, reported in Fig. 8c, the test set is classified as in Fig. 8b.

C. MNIST dataset

As a concluding example, it is interesting to show the application of the proposed quantum neuron model to the classification of the MNIST dataset, composed of 70000 grayscale images of digits ranging from zero to nine. A selection of sample images extracted from the given dataset are reported in Fig. 9. We limit ourselves to the binary problem of correctly classifying the images of zeros and ones. Since each image in the MNIST dataset is composed of 28×28 pixels, which is clearly not in the suited

[4] Looking at the form of the activation function in (6), it can be seen that it only depends on the *differences* between the parameters. Thus, fixing one of the parameters to a constant value can be thought as just choosing a reference point.

Accepted Manuscript

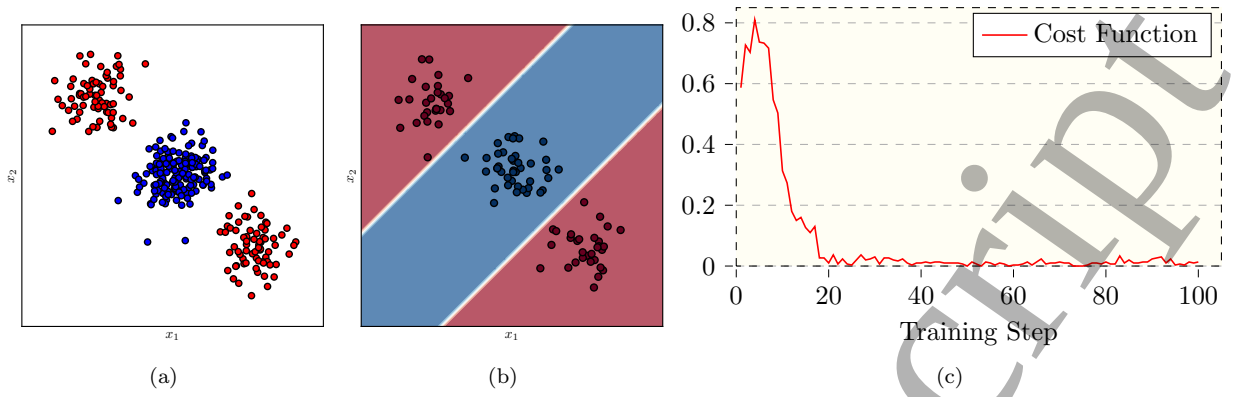


FIG. 7: Classification of two dimensional data. (a) Input data used as training set. (b) Test set and decision boundary implemented by the quantum neuron at the end of the learning procedure. The threshold used is $t = 0.95$. (c) Optimization with the SPSA optimizer run on the QASM Simulator.

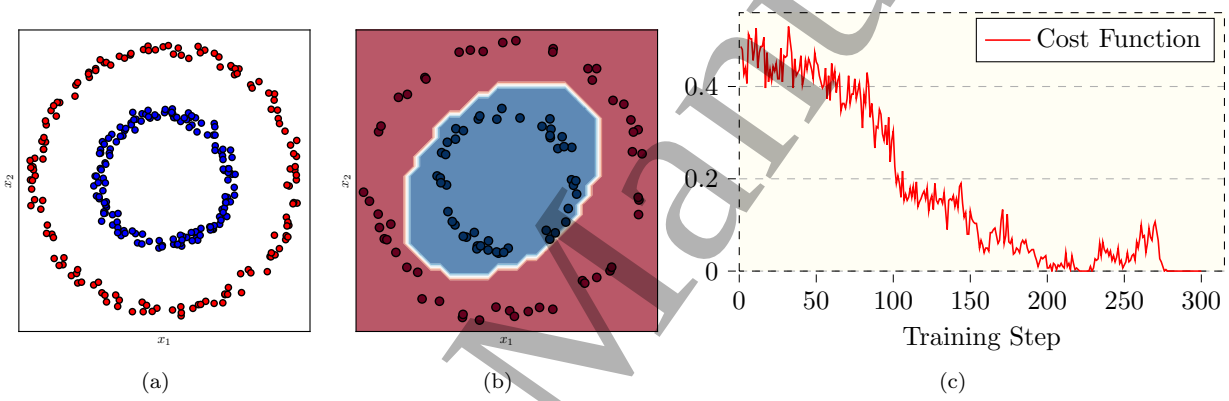


FIG. 8: Classification of two dimensional circles. (a) Input data used as the training set. (b) Test set and decision boundary implemented by the neuron at the end of the learning procedure. The threshold used in this example is $t = 0.95$, and the bias $b = 0.25$. (c) Optimization by the SPSA optimizer run on the QASM Simulator. The supervised learning procedure was performed with a batch examples of size 20, which explains why the error is not smooth but presents several spikes.

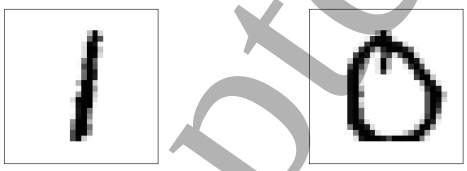


FIG. 9: Examples of images drawn from the MNIST dataset.

form $2^{n/2} \times 2^{n/2}$ required to be encoded on the quantum state of an artificial neuron with $N = 2^n$ input data, we modify the images by adding a number of white redundant pixels, such that the processed images have 32×32 pixels. A quantum artificial neuron with $n = 10$ qubits can thus be used to encode the input images. Here we limit our analysis to checking whether the activation function introduced in Eq. (6) is sufficient to discriminate between the encoded images of zeros and ones. With this goal in mind, we fix the weight vector of the artificial neuron to a sample "one" selected from the MNIST dataset, and then proceed to the classification with the remaining input images. Using a threshold of $t = 0.85$ in Eq. (17), the cost function evaluated on a test set of $m = 2060$ images amounts to $\mathcal{L} \sim 0.02$, which in turn means an accuracy $\sim 98\%$. In Fig. 10 it is shown a matrix containing the fidelities of the of some zeros and ones from the MNIST dataset evaluated with the activation function of the quantum neuron. According to the artificial neuron, the "ones" are more similar among each other with respect to the "zeros": indeed it seems that the neuron specializes in recognizing the "1" shape as the fidelity values are higher in the corresponding region of Fig. 10. Even if classical machine learning techniques can yield a classification accuracy above 99%, the present results

Accepted Manuscript

show a remarkable degree of precision, also considering that in this particular example just a single quantum neuron has been used for the classification. In addition to this strategy, we also tried a *pooling* procedure, in which each image in the MNIST dataset is first reduced to a 4×4 image by means of a *mean pooling* filter, and then classified by the neuron. After the learning, the neuron reaches a best accuracy around 80%. Nonetheless, these preliminary results show the potential of the activation function implemented by the quantum neuron to be used for recognition of complex patterns, such as numerical digits. Our quantum neuron model performs well when compared with other proposed quantum algorithms for the classification of the MNIST dataset. In fact, alternative algorithms have been proposed for this task, some of them using a hybrid classical-quantum approach, such as leveraging well established classical pre/post processing of data through classical machine learning techniques [38, 39]. These hybrid approaches may yield higher (although comparable) classification accuracy when compared to our quantum neuron model. However, we emphasize that in our case the artificial neuron model is fully quantum in nature. When compared to other works [31, 40] using only quantum resources and reporting accuracies of the order of 85% to 98%, our model seems to yield comparable or better results.

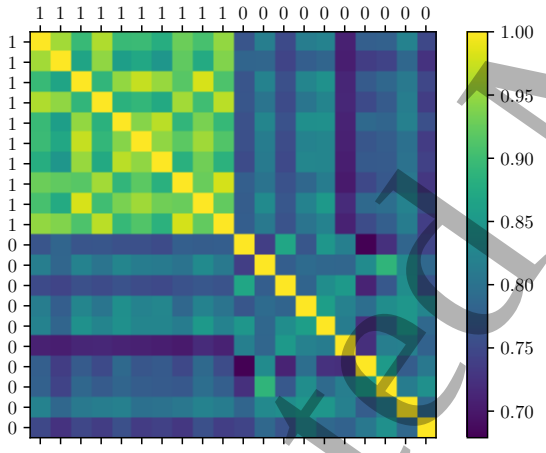


FIG. 10: Matrix containing the fidelities of some samples of "one" and "zero" images taken from the MNIST dataset, evaluated with the activation function in Eq. (6) and implemented by our quantum neuron model.

V. CONCLUSIONS

We have reported on a novel quantum algorithm allowing to implement a generalized perceptron model on a qubit-based quantum register. This quantum artificial neuron accepts and analyzes continuously valued input data. The proposed algorithm is translated into a quantum circuit model to be readily run on existing quantum hardware. It takes full advantage of the exponentially large Hilbert space available to encode input data on the phases of large superposition states, known as locally maximally entanglable (LME). These LME states can be constructed with a bottom-up approach, by imprinting each single phase separately. However, it should be stressed that alternative and possibly more efficient strategies could directly yield such states as ground states of suitable Hamiltonians, or as stationary states from dissipative processes [32]. The proposed continuously valued quantum neuron proves to be a good candidate for classification tasks of linearly non-separable two dimensional data, mostly related to pattern recognition tasks involving grayscale images. In this regard, thanks to the phase encoding, the neuron can leverage a built-in "color translational" invariance, as well as significant noise resilience. In particular, the activation function implemented by the quantum neuron yields very high accuracy in the order of 98% when used to discriminate between images of zeros and ones from the MNIST dataset, thus indicating the ability to distinguish also complex patterns. A further step would be to consider multiple layers of connected quantum neurons to build a continuous quantum feed-forward neural network. In addition, it would be interesting to study the application of phase encoding to other quantum machine learning techniques, such as quantum autoencoders. An important future direction would also be to design approximate methods to perform the weight unitary transformation in a way which scales more favorably with the number of encoding qubits: this could be achieved, for example, by training suitable variational or adaptive quantum circuits.

ACKNOWLEDGMENTS

This research was partly supported by the Italian Ministry of Education, University and Research (MIUR) through the "Dipartimenti di Eccellenza Program (2018-2022)", Department of Physics, University of Pavia, the PRIN Project "INPhoPOL" and the Quantera project QuICHE. We acknowledge ENI S.p.A. for having partially contributed to this project through the Framework agreement with Università di Pavia. We acknowledge use of the IBM Quantum Experience for this work. The views expressed are those of the authors and do not reflect the official policy or position of IBM company or the IBM-Q team.

DATA AVAILABILITY

All the data, codes and simulations that support the findings of this study may be available from the corresponding author upon reasonable request.

Accepted Manuscript

Appendix A: Modulus square

The squared modulus of the collection of complex numbers $\{z_i = r_i e^{i\gamma_i} \in \mathcal{C} |\ i = 1, \ldots, N\}$, is given as

$$\begin{aligned}\left|\sum_{i=1}^{N} z_i\right|^2 &= \left(\sum_{i=1}^{N} z_i\right)\left(\sum_{j=1}^{N} z_j^*\right) = \sum_{i,j=1}^{N} z_i z_j^* \ , \\ &= \sum_{i=j}^{N} |z_i|^2 + \sum_{i\neq j}^{N} r_i r_j e^{i(\gamma_i-\gamma_j)} \ , \\ &= \sum_{i=j}^{N} r_i^2 + 2\sum_{i<j}^{N} r_i r_j \cos(\gamma_j - \gamma_i) \ , \end{aligned} \tag{A1}$$

where in the last line the following relation has been applied

$$e^{ix} + e^{-ix} = 2\cos(x) \ . \tag{A2}$$

Setting $r_i = 1/N$ and $\gamma_i = \theta_i - \phi_i$, respectively, we finally get:

$$\left|\sum_{i=1}^{N} \frac{e^{i(\theta_i-\phi_i)}}{N}\right|^2 = \frac{1}{N} + \frac{2}{N^2}\sum_{i<j}^{N} \cos((\theta_j - \phi_j) - (\theta_i - \phi_i)) \ , \tag{A3}$$

which correctly reduces to Eq. (6) in the main text, upon substituting $N = 2^n$ and shifting the summation indices to start from zero.

Appendix B: Noise resilience

Consider an input vector $\boldsymbol{\theta}$ equal to the weight vector $\boldsymbol{\phi} = \boldsymbol{\theta}$. Now, suppose that the input is corrupted and transformed into $\boldsymbol{\theta}' = \boldsymbol{\theta} + \boldsymbol{\Delta}$, with $\Delta = (\Delta_0, \Delta_1, \ldots, \Delta_{2^n-1})$. In this case, the activation function in Eq. (6) reduces to

$$f(\boldsymbol{\Delta}) = \frac{1}{2^n} + \frac{1}{2^{2n-1}}\sum_{i<j}^{2^n-1} \cos(\Delta_j - \Delta_i) \ . \tag{B1}$$

Assuming that the noise values Δ_i are given by the uniform distribution:

$$p(\Delta_i) = \frac{1}{a} \quad \text{for } \Delta_i \in \left[-\frac{a}{2}, \frac{a}{2}\right] \ , \quad a \in \mathbb{R} \ , \tag{B2}$$

it is possible to evaluate the *average* activation function:

$$\begin{aligned}\langle f(\boldsymbol{\Delta})\rangle &= \frac{1}{2^n} + \frac{1}{2^{2n-1}}\sum_{i<j}^{2^n-1} \langle\cos(\Delta_j - \Delta_i)\rangle \\ &= \frac{1}{2^n} + \frac{1}{2^{2n-1}}\frac{2^n(2^n-1)}{2}\langle\cos(\Delta_j - \Delta_i)\rangle \end{aligned} \tag{B3}$$

where in the last line it is implicitly assumed that $\langle\cos(\Delta_j - \Delta_i)\rangle$ is the same for all i, j. The averaging then yields

$$\begin{aligned}\langle\cos(\Delta_j - \Delta_i)\rangle &= \int_{-\frac{a}{2}}^{\frac{a}{2}}\int_{-\frac{a}{2}}^{\frac{a}{2}} \cos(\Delta_j - \Delta_i)\frac{d\Delta_i d\Delta_j}{a^2} \\ &= 2\left(\frac{1-\cos(a)}{a^2}\right) \ , \end{aligned} \tag{B4}$$

and substituting back into (B3), we eventually get:

$$\langle f(\boldsymbol{\Delta})\rangle = \frac{1}{2^n} + \frac{2^n-1}{2^{n-1}}\left(\frac{1-\cos(a)}{a^2}\right) \ . \tag{B5}$$

Consider now the case where the input and weight vectors are different, $\boldsymbol{\theta} \neq \boldsymbol{\phi}$. The question is how much does the activation function change, if the input is corrupted by the presence of noise. As before, considering an input $\boldsymbol{\theta}' = \boldsymbol{\theta} + \boldsymbol{\Delta}$, the activation function reads:

$$f(\boldsymbol{\theta}, \boldsymbol{\phi}, \boldsymbol{\Delta}) = \frac{1}{2^n} + \frac{1}{2^{2n-1}}\sum_{i<j}^{2^n-1} \cos(A_{ij} + D_{ij}) \ . \tag{B6}$$

with $A_{ij} = (\theta_j - \phi_j) - (\theta_i - \phi_i)$, and $D_{ij} = \Delta_j - \Delta_i$. Since $\cos(A_{ij} + D_{ij}) = \cos(A_{ij})\cos(D_{ij}) + \sin(A_{ij})\sin(D_{ij})$, and $\langle\sin(D_{ij})\rangle = 0$ (using the probability distribution in Eq. (B2)), it finally results in

$$\langle f(\boldsymbol{\theta}, \boldsymbol{\phi}, \boldsymbol{\Delta})\rangle = \frac{1}{2^n} + \frac{D}{2^{2n-1}}\sum_{i<j}^{2^n-1} \cos(A_{ij}) \ . \tag{B7}$$

with $D = 2\left(\frac{1-\cos(a)}{a^2}\right)$.

A more realistic noise model would consist of a Gaussian distribution centred in zero with width $\sigma = a/2$

$$p(\Delta_i) = \frac{1}{\sqrt{2\pi(a/2)^2}} e^{-\frac{\Delta_i^2}{2(a/2)^2}} \tag{B8}$$

in this case, it can be shown that

$$\begin{aligned}\langle\cos(\Delta_i - \Delta_j)\rangle &= \frac{2}{\pi a^2}\int_{-\infty}^{\infty}\int_{-\infty}^{\infty} e^{-\sqrt{2}(\Delta_i+\Delta_j)^2/a^2} \times \\ &\qquad \times \cos(\Delta_i - \Delta_j) d\Delta_i d\Delta_j \\ &= e^{-a^2/4} \ . \end{aligned} \tag{B9}$$

This is comparable to the uniform distribution noise model, but less effective in terms of noise resilience, which is due to the Gaussian tails having the net effect of lowering the mean activation. A narrower Gaussian would yield better results, of course. Nonetheless, this qualitative behaviour proves the quantum neuron to have some internal degree of noise resilience.

Appendix C: Alternative schemes for the implementation of U_i and U_w operations

Several strategies can be envisioned to reduce the computational complexity of the proposed quantum algorithm. As an example, the loading of input data could

Accepted Manuscript

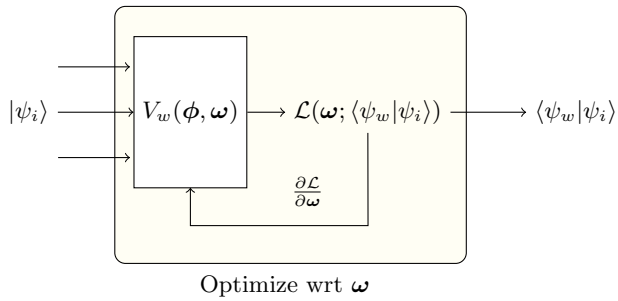


FIG. 11: Scheme of the variational circuit for the implementation of the U_w quantum operation.

effectively be replaced by a direct call to a quantum memory, such as a qRAM [41]. In this case, the information to be analyzed would be directly stored in the form of quantum states coming, e.g. from quantum internet applications or quantum simulators. Alternatively, one could make use of some specific properties of the LME states used in the main text for the encoding. Indeed, let U_ψ be the unitary operation whose action is to create an LME state from a blank register, i.e $U_\psi |0\rangle = |\psi\rangle$. It is then easy to check that, for $W_k = U_\psi Z_k U_\psi^\dagger$, where Z_k is the σ_z Pauli operator acting on the $k-$th qubit, it holds that $W_k |\phi\rangle = |\phi\rangle \ \forall k$ if and only if $|\phi\rangle = |\psi\rangle$. This means that the operators $\{W_1, W_2, \ldots, W_n\}$ stabilize the state $|\psi\rangle$. Depending on the values of the phases $\{\alpha_i\}_{i=0}^{2^{n-1}}$, these operators $\{W_k\}_{k=0}^{n-1}$ may be quasi-local, meaning that they only act on a smaller subsystem of the whole $n-$qubit register. In this case, it can be shown [32, 42] that there exists a quantum dissipative process for which $|\psi\rangle$ is the only stationary state. Of course, this property strongly depends on the nature of the phases α_i of the target LME state, i.e., correlations in the phases are directly related to a specific preparation scheme. However, it might be the case that for some special classes of incoming inputs, a clear a priori correlation exists between the phases, which would allow to replace the "brute force" approach used in the main text with a more efficient preparation scheme. Finally, it is worth mentioning that more general strategies to load probability distributions and classical datasets on quantum states are known in the literature [43, 44], whose application could be investigated also in the present case.

Additional room for improvement could be represented by a more efficient implementation of the inner product operation U_w. In this case, instead of simply inverting the preparation circuit, one could devise a variational circuit optimized to output the desired result, see Eq. (6). For example the circuitry in Figure 11 could be used: where $V_w(\boldsymbol{\phi}, \boldsymbol{\omega})$ is an operator approximating the unitary U_w, depending on the variational parameters $\boldsymbol{\omega}$, and the weights of neuron $\boldsymbol{\phi}$; and $\mathcal{L}(\omega; \langle\psi_w|\psi_i\rangle)$ is a cost function evaluating the distance between the output from $V_w(\boldsymbol{\phi}, \boldsymbol{\omega})$ and the desired inner product $\langle\psi_w|\psi_i\rangle$. Optimizing the values for $\boldsymbol{\omega}$, could yield to an approximate yet efficient implementation for evaluating the inner product gate U_w. It is also interesting to notice that such optimization procedure could in principle be carried out in combination with a supervised learning approach in order to simultaneously train the value of the weight vector and the actual quantum circuit realization of the required operation.

Finally, if an efficient preparation scheme exists for both the quantum input state, $|\psi_i\rangle$, and the quantum weight state, $|\psi_w\rangle$, their inner product could also be evaluated by means of specialized algorithms, such as the SWAP test or the Bell basis algorithm [45].

[1] F. Arute, K. Arya, R. Babbush, *et al.*, Nature **574**, 505–510 (2019).
[2] J. Preskill, Quantum **2**, 79 (2018).
[3] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, Nature **549**, 195 (2017).
[4] V. Dunjko and H. J. Briegel, Reports on Progress in Physics **81**, 074001 (2018).
[5] M. Benedetti, E. Lloyd, S. Sack, and M. Fiorentini, Quantum Science and Technology **4**, 043001 (2019), publisher: IOP Publishing.
[6] M. Schuld, I. Sinayskiy, and F. Petruccione, Physics Letters A **379**, 660 (2015).
[7] N. Wiebe, A. Kapoor, and K. M. Svore, arXiv:1602.04799 [quant-ph, stat] (2016).
[8] Y. Cao, G. G. Guerreschi, and A. Aspuru-Guzik, arXiv:1711.11240 [quant-ph] (2017).
[9] F. Tacchino, C. Macchiavello, D. Gerace, and D. Bajoni, npj Quantum Information **5**, 26 (2019).
[10] E. Torrontegui and J. J. Garcia-Ripoll, EPL (Europhysics Letters) **125**, 30004 (2019).
[11] L. B. Kristensen, M. Degroote, P. Wittek, A. Aspuru-Guzik, and N. T. Zinner, arXiv:1907.06269 [cond-mat, physics:quant-ph] (2019).
[12] N. Killoran, T. R. Bromley, J. M. Arrazola, M. Schuld, N. Quesada, and S. Lloyd, Phys. Rev. Research **1**, 033063 (2019).
[13] V. Havlíček, A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, and J. M. Gambetta, Nature **567**, 209 (2019).
[14] M. Schuld, M. Fingerhuth, and F. Petruccione, EPL (Europhysics Letters) **119** (2017).
[15] M. Schuld and N. Killoran, Physical Review Letters **122**, 040504 (2019).
[16] J. Romero, J. P. Olson, and A. Aspuru-Guzik, Quantum Science and Technology **2**, 045001 (2017).
[17] L. Lamata, U. Alvarez-Rodriguez, J. D. Martín-Guerrero, M. Sanz, and E. Solano, Quantum Science and Technology **4**, 014007 (2018).
[18] M. Henderson, S. Shakya, S. Pradhan, and T. Cook, Quantum Machine Intelligence **2**, 1 (2020).

Accepted Manuscript

[19] I. Cong, S. Choi, and M. D. Lukin, Nature Physics **15**, 1273 (2019).
[20] M. H. Amin, E. Andriyash, J. Rolfe, B. Kulchytskyy, and R. Melko, Phys. Rev. X **8**, 021050 (2018).
[21] T. Xiao, D. Bai, J. Fan, and G. Zeng, Phys. Rev. A **101**, 032304 (2020).
[22] I. Kerenidis, J. Landman, and A. Prakash, Quantum algorithms for deep convolutional neural networks (2019), https://arxiv.org/abs/1911.01117.
[23] M. Schuld, I. Sinayskiy, and F. Petruccione, Quantum Information Processing **13**, 2567 (2014).
[24] J. M. Zurada, *Introduction to Artificial Neural Systems* (West Group, 1992).
[25] R. Rojas, *Neural Networks: A Systematic Introduction* (Springer, 1996).
[26] W. S. McCulloch and W. Pitts, The bulletin of mathematical biophysics **5**, 115 (1943).
[27] F. Rosenblatt, *The perceptron, a perceiving and recognizing automaton Project Para* (Cornell Aeronautical Laboratory, 1957).
[28] P. Li and H. Xiao, Applied Intelligence **40**, 107 (2013).
[29] P. Q. Le, F. Dong, and K. Hirota, Quantum Information Processing **10**, 63 (2010).
[30] J. I. Latorre, Image compression and entanglement (2005), arXiv:quant-ph/0510031.
[31] E. Farhi and H. Neven, Classification with quantum neural networks on near term processors (2018), https://arxiv.org/abs/1802.06002.
[32] C. Kruszynska and B. Kraus, Phys. Rev. A **79**, 052304 (2009).
[33] Y. LeCun, Y. Bengio, and G. Hinton, Nature **521**, 436–444 (2015).
[34] M. A. Nielsen and I. L. Chuang, *Quantum computation and quantum information*, 10th ed. (Cambridge University Press, 2010).
[35] F. Tacchino, P. Barkoutsos, C. Macchiavello, I. Tavernelli, D. Gerace, and D. Bajoni, arxiv:1912.12486 (2019).
[36] H. Abraham *et al.*, Qiskit: An open-source framework for quantum computing (2019).
[37] J. C. Spall, IEEE Transactions on Aerospace and Electronic Systems **34**, 817 (1998).
[38] A. Asfaw, L. Bello, *et al.*, Learn quantum computation using qiskit (2020), Hybrid quantum-classical Neural Networks with PyTorch and Qiskit.
[39] I. Kerenidis and A. Luongo, Classification of mnist dataset with quantum slow feature analysis (2018), https://arxiv.org/abs/1805.08837.
[40] M. Broughton, G. Verdon, *et al.*, arXiv:2003.02989 [cond-mat, physics:quant-ph] (2020), MNIST Classification Tutorial.
[41] V. Giovannetti, S. Lloyd, and L. Maccone, Phys. Rev. Lett. **100**, 160501 (2008).
[42] B. Kraus, H. P. Büchler, S. Diehl, A. Kantian, A. Micheli, and P. Zoller, Phys. Rev. A **78**, 042307 (2008).
[43] L. Grover and T. Rudolph, Creating superpositions that correspond to efficiently integrable probability distributions (2002), arXiv:quant-ph/0208112.
[44] M. Schuld and F. Petruccione, *Supervised Learning with Quantum Computers* (Springer International Publishing, 2018).
[45] L. Cincio, Y. Subaşı, A. T. Sornborger, and P. J. Coles, New Journal of Physics **20**, 113022 (2018).

Accepted Manuscript